

7º

Bucles — repeticiones inteligentes en los algoritmos

Guía 17-7-TIC

LO QUE TE LLEVAS HOY

Un **algoritmo que use al menos 2 bucles** (uno de tipo *repetir N veces*, otro de tipo *repetir hasta que*). Tema sugerido: “Tejer una pulsera de 30 vueltas”, “Saludar a los 20 invitados a una fiesta”, “Lavarse los dientes (cepillar arriba, abajo, izquierda, derecha)”. Más **su diagrama de flujo** (con el bucle dibujado correctamente). Más **tabla comparativa** de los 2 tipos de bucles en el cuaderno.

ESTA GUÍA ES DE

Nombre completo

Grupo

Fecha

Apertura: Bucles — repeticiones inteligentes en los algoritmos

Saber ancestral

Doña Sofía la tejedora del Valle del Cauca repetía el mismo patrón 200 veces para tejer una mochila. En la vereda La Plata, las tejedoras de canastos del Cauca, las hamaqueras del Pacífico, las costureras de Buga tenían un saber compartido: *el patrón se repite*. Cuando doña Sofía empezaba una mochila, sabía que tenía que ejecutar la secuencia “*levantar hebra, pasarla sobre 2, pasarla por debajo de 1*” **exactamente 200 veces**. ¿Contaba mentalmente? Sí. ¿Se aburría? No — la repetición rítmica era *meditativa*. ¿Se equivocaba? A veces, y entonces destejía hasta el último punto correcto y volvía a empezar. Esa estructura — “*repetir N veces*” o “*repetir hasta lograr X*” — es exactamente lo que en programación llamamos **bucle** o **ciclo**. Sin bucles, una mochila Wayuu tomaría escribir 200 instrucciones separadas. **Con bucles, una instrucción se ejecuta automáticamente N veces**. Es la elegancia algorítmica que las tejedoras ya conocían.

Saber contemporáneo

Un **bucle** (o *ciclo*, o *loop* en inglés) es una estructura que **repite una o varias acciones hasta que se cumple una condición**. Es el tercer componente fundamental de la programación (después de variables y condicionales). **Hay 2 tipos básicos. (1) Bucle “repetir N veces”** (*for*): se repite un número fijo de veces. Ejemplo: *REPETIR 10 VECES: dar pasos*. Útil cuando sabes exactamente cuántas repeticiones quieres. **(2) Bucle “repetir hasta que”** (*while*): se repite mientras una condición es verdadera. Ejemplo: *REPETIR HASTA QUE peso = 100kg: comer*. Útil cuando no sabes cuántas repeticiones, pero sabes cuándo parar. **Los bucles son lo que hace que un programa pueda procesar muchos datos sin escribir mil veces lo mismo**: leer todos los correos, mover todos los archivos, sumar todas las notas. Sin bucles, los programas serían imposibles de escribir.

Pregunta-puente

Si tuvieras que escribir un programa para “saludar a los 30 estudiantes de la clase, uno por uno, llamándolos por su nombre”, ¿**escribirías 30 instrucciones separadas** o **usarías un bucle**?

Saber y saber hacer

Al terminar la clase: (1) podrás **identificar** los 2 tipos de bucles; (2) sabrás **aplicar** cada tipo a su situación; (3) podrás **evaluar** si un algoritmo necesita bucle o no; (4) habrás **creado** un algoritmo con 2 bucles + diagrama.

Autor	Dr. Álvaro Cárdenas Orozco
DBA	Construye algoritmos y diagramas de flujo para resolver problemas paso a paso (MEN, T&I 7°)
Referentes	Saber ancestral del tejedor · Pensamiento computacional · Scratch
Producto final	Un algoritmo que use al menos 2 bucles (uno de tipo <i>repetir N veces</i> , otro de tipo <i>repetir hasta que</i>). Tema sugerido: “Tejer una pulsera de 30 vueltas”, “Saludar a los 20 invitados a una fiesta”, “Lavarse los dientes (cepillar arriba, abajo, izquierda, derecha)”. Más su diagrama de flujo (con el bucle dibujado correctamente). Más tabla comparativa de los 2 tipos de bucles en el cuaderno.

→ Hoy haces 4 cosas: identificas repeticiones en situaciones reales, aprendes los 2 tipos de bucles, escribes algoritmo con 2 bucles, dibujas el diagrama.

Ruta MILC: así vamos a aprender

1

Escucha
Observo, escucho
y pregunto.

2

Entender
Sistematizo
y ordeno.

3

Hacer
Construyo y aplico.

4

Cierre
Evalúo y reflexiono.

→ Empezamos viendo qué pasaría sin bucles (caos).

Estación 1 de 3 · Escucha

Escena para escuchar

Actividad 1 · ANALIZA — ¿Cuántas instrucciones sin bucle? (10 min · individual). Imagina este problema: *tienes que dar la bienvenida a los 30 estudiantes nuevos de 7°. A cada uno: “Hola, bienvenido a la clase”.* **En el cuaderno responde:** (1) Sin usar bucles, ¿cuántas instrucciones escribirías?. (2) ¿Cuánto te tomaría escribirlas?. (3) ¿Qué pasa si en lugar de 30 son 300 estudiantes?. (4) ¿Crees que hay una manera más inteligente?.

MARCA CUANDO LO HAYAS HECHO

- ☐ Pensé en el problema sin bucle.
- ☐ Calculé cuántas instrucciones serían.
- ☐ Reconocí que escalar es problema sin bucles.

Lo que va al cuaderno (Actividad 1 · ANALIZA). Título: «Actividad 1 — 30 saludos sin bucle». **Formato:** 4 respuestas cortas. **Extensión:** 5-6 líneas. **Sabes que terminaste cuando** ves que sin bucles los programas serían inmanejables.

→ Después aprendes los 2 tipos + el contador como variable amiga.

Estación 2 de 3 · Entender

Lo que vas a entender

La intuición es correcta: **sin bucles, los programas se vuelven imposibles**. Ahora aprende los 2 tipos de bucles y cómo se escriben.

- Bucle tipo 1 — “Repetir N veces” (for).** Cuando *sabes exactamente cuántas veces* quieres repetir. Estructura: *REPETIR [N] VECES: [acción]*. Ejemplo del saludo: *REPETIR 30 VECES: decir “Hola, bienvenido”*. Eso es 1 instrucción que ejecuta 30 veces. En programación a veces se usa una **variable contador** que va de 1 a N: *PARA cada estudiante DESDE 1 HASTA 30: decir “Hola estudiante [contador]”*. La variable contador toma valores 1, 2, 3, ..., 30 sucesivamente.
- Bucle tipo 2 — “Repetir hasta que” (while).** Cuando *no sabes cuántas veces* pero sabes la condición de parada. Estructura: *REPETIR HASTA QUE [condición]: [acción]*. Ejemplo: *REPETIR HASTA QUE puntaje $\geq$ 100: jugar un nivel*. Se repite mientras puntaje sea menor que 100. Cuando puntaje llega a 100 (o más), el bucle termina. Otra variante: *MIENTRAS [condición es verdadera]: [acción]* — igual de común, lógica equivalente.
- El contador como variable amiga del bucle.** Casi todo bucle usa una **variable contador**: número que se incrementa cada vuelta del bucle. Ejemplo de tejido: *vueltaActual = 0; REPETIR HASTA QUE vueltaActual = 200: tejer una vuelta; vueltaActual = vueltaActual + 1*. Cada iteración: teje 1 vuelta y suma 1 al contador. Cuando llega a 200, termina. El contador es lo que conecta variables (S5) con bucles (S7).
- El bucle infinito (cuidado).** Un bucle que NUNCA termina porque la condición nunca se vuelve falsa. Ejemplo MAL: *REPETIR HASTA QUE 5 > 10: jugar* — esto es siempre falso, el bucle no para nunca, el programa se congela. **Cómo evitarlo:** (a) verificar que la condición se pueda volver verdadera en algún momento; (b) verificar que algo cambia adentro del bucle (incrementar contador, modificar variable). En Scratch y otros lenguajes, los bucles infinitos hacen que el programa se trabe.

2 tipos de bucles + contador + cuidado infinito

Tipo 1 (for): REPETIR [N] VECES: [acción].

Tipo 2 (while): REPETIR HASTA QUE [condición]: [acción].

Contador: variable que cuenta las iteraciones ($\text{vueltaActual} = 0$; $\text{vueltaActual} = \text{vueltaActual} + 1$).

Cuidado bucle infinito: verificar que la condición se pueda cumplir.

Errores comunes y su corrección

Errores frecuentes con bucles. (1) *Olvidar incrementar el contador* — bucle infinito porque nunca cambia la condición. (2) *Condición que nunca se vuelve verdadera* — bucle infinito. “REPETIR HASTA QUE $0 = 1$ ” no termina nunca. (3) *Usar bucle cuando no se necesita* — si solo es 1 vez, no necesitas bucle. (4) *Usar tipo for cuando debería ser while (o viceversa)* — depende de si conoces el número exacto o solo la condición. (5) *Anidar bucles sin necesidad* — 5 bucles dentro de bucles se vuelven inentendibles. Mejor descomponer.

Lo que va al cuaderno (Actividad 2 · EXPLICA). Título: «Actividad 2 — 2 tipos de bucles». **Formato:** tabla comparativa de 4 filas, 3 columnas (criterio, FOR, WHILE). Filas: *cuándo se usa, ejemplo de la vida real, estructura escrita, cómo se ve en diagrama*. **Extensión:** 4 filas. **Sabes que terminaste cuando** podrías escoger el tipo correcto de bucle para cualquier situación.

→ Con todo claro, escribes tu algoritmo con bucles + diagrama.

Estación 3 de 3 · Hacer

Cómo vas a construir tu producto

Actividad 3 · APLICA — Algoritmo con 2 bucles + diagrama (30 min · individual). Vas a escribir un algoritmo que use **2 bucles distintos** (uno de cada tipo) y dibujarlo en diagrama de flujo.

1

Paso 1 — Escoge el tema del algoritmo. Tiene que ser algo que requiera *2 tipos de repetición*: una *N veces* (sabes cuántas) y una *hasta que* (no sabes cuántas, sabes cuándo parar). Sugerencias: “Tejer una pulsera de 30 vueltas pero hasta que se acabe el hilo” (for por las 30 vueltas, while por el hilo). “Saludar a los 20 invitados de una fiesta y hasta que se acaben los bocaditos” (for por los 20, while por los bocaditos). “Estudiar 5 ejercicios pero hasta que entienda el tema”.

2

Paso 2 — Identifica las variables y el contador. Lista las variables: *cantidadVueltas* (número), *hiloRestante* (número), *contadorVueltas* (número, empieza en 0). Identifica cuál es el contador para cada bucle.

3 Paso 3 — Escribe el algoritmo. Estructura sugerida: *INICIO. Variables: cantidadVueltas = 30, hiloRestante = 100, contadorVueltas = 0. BUCLE 1 (for): PARA contadorVueltas DESDE 1 HASTA 30: tejer 1 vuelta; hiloRestante = hiloRestante - 1. BUCLE 2 (while): MIENTRAS hiloRestante > 0: tejer 1 vuelta más; hiloRestante = hiloRestante - 1. FIN.* Adapta a tu tema.

4 Paso 4 — Dibuja el diagrama de flujo. Los bucles se representan así en diagrama: **(a)** óvalo INICIO. **(b)** Para el bucle “repetir N veces”: rombo “¿contador < N?”, flecha SÍ va al rectángulo de la acción + incremento del contador, flecha NO va al siguiente paso. La acción y el incremento vuelven al rombo (creando el ciclo visual). **(c)** Para el bucle “repetir hasta que”: rombo “¿condición?”, flecha SÍ va al rectángulo de la acción y vuelve al rombo, flecha NO sale del bucle. **(d)** óvalo FIN.

5 Paso 5 — Tabla comparativa de los 2 bucles. En una esquina del cuaderno, dibuja la tabla de la S2 (Actividad 2) con los 2 tipos. Marca cuál usaste en cada bucle de tu algoritmo. Esto consolida el conocimiento.

6 Paso 6 — Verificación de bucle no infinito + reflexión + firma. Revisa tu algoritmo: ¿el bucle for tiene condición que se cumple eventualmente? (sí, porque el contador llega a 30). ¿el bucle while tiene condición que se vuelve falsa eventualmente? (sí, porque hiloRestante disminuye). Si los 2 cumplen, tu algoritmo termina. Escribe la reflexión: “El tipo de bucle que más me costó entender fue _____. Si no hubiera bucles en programación, escribir un algoritmo de 100 pasos sería _____”. Firma con nombre y fecha.

Antes de cerrar la S7

- ☐ Mi algoritmo tiene 2 bucles, uno de cada tipo.
- ☐ El contador del bucle for está identificado.
- ☐ El bucle while tiene condición que se vuelve falsa eventualmente.
- ☐ El diagrama de flujo muestra los 2 bucles visualmente.
- ☐ La tabla comparativa for vs while está completa.
- ☐ Verifiqué que NO hay bucle infinito en mi algoritmo.

Plantilla de trabajo

Estructura. Paso 1: tema del algoritmo (que requiera 2 tipos de repetición). Paso 2: variables + contador. Paso 3: algoritmo con bucle for + bucle while. Paso 4: diagrama de flujo. Paso 5: tabla comparativa. Paso 6: verificación + reflexión + firma.

Lo que va al cuaderno (Actividad 3 · APLICA). Título: «Actividad 3 — Algoritmo con 2 bucles». **Formato:** variables + algoritmo + diagrama + tabla + verificación. **Extensión:** 1 página y media. **Sabes que terminaste cuando** podrías ejecutar mentalmente tu algoritmo y verificar que termina (no es infinito).

→ El producto es algoritmo + diagrama + tabla comparativa.

Producto de la sesión

Algoritmo con 2 bucles + diagrama · for y while aplicados · sin bucle infinito.

Criterios de calidad

(1) El algoritmo tiene 1 bucle for (repetir N veces) bien aplicado. (2) El algoritmo tiene 1 bucle while (repetir hasta que) bien aplicado. (3) El diagrama de flujo muestra los 2 bucles visualmente. (4) El contador del bucle for está identificado. (5) Se verificó que ningún bucle es infinito.

→ Antes de salir, verifica que tu algoritmo NO entre en bucle infinito (que sí termine).

Evaluación liberadora · cinco dimensiones

Sentido de la evaluación

Evaluar no es solo revisar una nota. Es reconocer qué aprendí, qué cambió en mí, qué emociones manejé, cómo me relaciono con la ciudad y la comunidad, y qué le dejaré a quien viene detrás.

Mi reflexión liberadora en cinco dimensiones. Completa cada frase iniciada:

Dimensión	Mi respuesta (frame starter)	Algo que descubrí
1. Personal	Lo que más cambió en mí fue _____	_____
2. Emocional	La emoción más fuerte fue _____ y la regulé _____	_____
3. Ciudadana	En democracia esto importa porque _____	_____
4. Local (Cartago)	En mi barrio sería útil para _____	_____
5. Intergeneracional	A mi abuela/abuelo le diría _____	_____

Para la próxima sustentación. Vuelve a esta tabla en seis meses. Verás que las respuestas cambian — no porque lo aprendido cambie, sino porque tú cambias. La evaluación liberadora es una conversación contigo a través del tiempo.

→ Cerramos con 3 voces sobre por qué saber repetir bien es virtud antigua.

Triángulo de pensamiento · cierre filosófico

Dussel · lente del nosotros

“La repetición no es aburrida cuando entiendes su sentido. La tejedora tejó 200 vueltas porque cada vuelta importaba.”

Hay una idea moderna de que la *repetición es aburrida* y debe automatizarse. Es cierto a medias: **lo que el computador puede repetir automáticamente, no debes repetirlo a mano**. Pero algunas repeticiones son meditativas y formativas: doña Sofía tejiendo, el atleta practicando, el músico ensayando. Los bucles automatizan las repeticiones mecánicas para que tú te concentres en las repeticiones formativas.

¿Qué repeticiones de mi vida son mecánicas (deberían automatizarse) y cuáles son formativas (vale la pena repetir conscientemente)?

Estoicismo · Marco Aurelio · cuidado interior

“Lo que se repite con criterio, perfecciona. Lo que se repite por inercia, embrutece. La diferencia está en la atención.”

Los bucles bien diseñados son como la disciplina diaria: hacen lo mismo cada vez, pero con propósito claro y condición de parada. Sin propósito (bucle infinito), repetir embrutece. Con propósito (condición clara), repetir perfecciona. Marco Aurelio escribía sus reflexiones cada noche — bucle con propósito, no inercia.

Las repeticiones de mi vida diaria, ¿tienen propósito claro o las hago por inercia?

Floridi · lente de la infoesfera

“Los bucles son lo que hizo posible la computación masiva. Sin bucles, no habría Google ni Wikipedia.”

Google indexa millones de páginas **usando bucles**: REPETIR por cada página: leer, analizar, indexar, pasar a la siguiente. Wikipedia tiene millones de artículos. Spotify procesa millones de canciones. Los bucles son lo que escaló la informática desde calculadoras simples hasta la infraestructura del mundo digital. Entenderlos en 7° es entender una pieza fundamental del mundo moderno.

¿Qué cosas del mundo digital que uso a diario serían imposibles sin bucles?

Nota para el docente. Las tres formulaciones del triángulo son la síntesis del autor de la guía, no citas textuales de los pensadores. Si vas a citarlos en clase, remite a la obra original.

Mi autoevaluación · marca una casilla por fila

Criterio	Lo logré	En proceso	Necesito apoyo
Comprensión del tema	☐ Explico las ideas con mis palabras.	☐ Reconozco las ideas, falta precisión.	☐ Necesito repasar lo básico.
Pensamiento computacional	☐ Usé los pilares al armar mi producto.	☐ Usé algunos pilares.	☐ Necesito releer la estación 2.
Aplicación y producto	☐ Mi producto cumple los criterios.	☐ Está armado, con ajustes pendientes.	☐ Aún está en construcción.
Reflexión 5 dimensiones	☐ Respondí cada una con honestidad.	☐ Respondí algunas con detalle.	☐ Mis respuestas son muy generales.
Triángulo de pensamiento	☐ Conecté las 3 voces con mi trabajo.	☐ Conecté al menos una.	☐ No las relaciono con mi proceso.

Hoy entendí que:

Mi compromiso para la próxima sesión es:

Cita de despedida · Marco Aurelio

“El obstáculo en el camino se vuelve el camino. Lo que estorba al camino se vuelve camino.”

Cada error de hoy, bien observado, se convierte en pieza del camino que pisarás mañana.

Nombre completo: _____

Fecha: _____ Curso/grupo: _____

Mi firma: _____

Para la próxima sesión. Esta semana, identifica 3 situaciones reales donde te tocaría usar bucles si fueras a automatizarlas. La próxima clase entramos a **Scratch**: abres tu primer programa con bloques. Vas a programar de verdad.